Worth the Lookup: Budgeted Relational Retrieval for Call-List Prioritization

Max Rodriguez **Stanford University** `maxrod@stanford.edu`

Project type: real-world application of GNNs (blog post + Colab). **Group:** Max Rodriguez (solo). Not shared with another course.
Code: `https://github.com/MaxLuisRodriguez/CS224W-fraud-detection-machine-learning-with-graphs/tree/milestone` (the version used for this report, tagged `milestone`; code map and run commands in the README).

1 Problem and motivation

Before each call, a sales rep's AI research agent can look up a prospect's recent activity, the accounts and events it touches, and other buying signals. Every lookup costs something: enrichment credits, LLM tokens, or latency before the dialer connects. No team researches every prospect deeply, so the real decision is *which lookups to make for the whole call list* within a budget. Sales data are linked tables, and the prospects on one list often touch the same accounts and events (Figure 1). That couples the prospects in two ways.

Shared information. Fetching one shared event informs every prospect connected to it, so its value is the sum of its effects on all of them.

A shared decision. A rep calls only the top K of the list, so a lookup matters only if it changes who lands in the top K, and that depends on everyone else's scores. Suppose five prospects score 0.80, 0.55, 0.53, 0.20 and 0.10, and the rep calls two. A lookup that moves the fourth prospect from 0.20 to 0.30 improves its prediction but changes no call. A lookup that moves the third prospect from 0.53 to 0.60 swaps who gets called.

This separates three goals that are easy to conflate: making each prediction more accurate (cross-entropy), ranking the whole list better (AUROC, average precision), and choosing the right top K (Precision@K). Researching prospects one at a time targets none of them for the list as a whole.

Research question. Given a call list of N prospects and a budget of B lookups, which lookups should an agent make so that a graph neural network (GNN) ranks the list best, and in particular picks the right top K?

Contributions. The project has three parts so far and a fourth after the milestone. (i) A relational GNN that scores a prospect when only part of its neighborhood has been retrieved, so a prediction exists at every budget. (ii) A lookup environment that prices retrieval. It has two actions, *pull* and *enrich*, each costing one unit, and a cache shared across the list, so a shared entity is paid for once. (iii) A comparison of lookup policies by their *budget frontier*, ranking quality against lookups per prospect, which shows how well each policy allocates a fixed budget. (iv) After the milestone, a learned policy that predicts each lookup's value for the whole list. The thread running through all four is the gap between three kinds of value: how much a row helps predict (*predictive information*), how much a lookup improves the list's predictions (*retrieval value*), and how much it changes who gets called (*decision value*). The milestone results show these can diverge.

Scope after mentor feedback. Our mentor advised completing the comparison of lookup policies on one dataset before expanding, since learning the lookup value adds significant complexity. Every result below is therefore on `rel-event`. `rel-f1` is used only to develop and test code, and `rel-avito` is a stretch goal. Two hypotheses that compared datasets are re-scoped to one: **(H1)** frontiers are concave: with a quarter of a lookup per prospect, a policy recovers most of the AUROC gap between zero lookups and full information; **(H2)** a model trained on randomly hidden rows scores

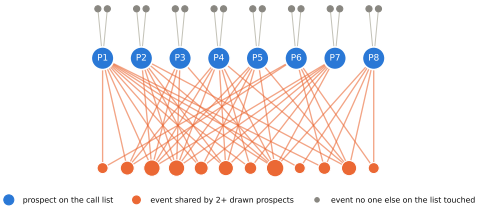


Figure 1: A real call list (`rel-event`, 2012-11-21): of a random 128-prospect list, the 8 prospects sharing the most events (blue), their 12 most-shared events (orange; larger = shared by more), and two unshared events each (grey). One orange lookup informs several prospects.

Table (node type)	Nodes	Foreign keys (edges)
`users`	37,143	–
`events`	2,459,811	creator (945)
`event_attendees`	8,430,002	event (8.43M), user (49.8K)
`event_interest`	14,978	event (14.1K), user (15.0K)
`user_friends`	30,386,403	user (30.0M), friend (215K)

Table 1: The `rel-event` graph: 41.3M nodes (rows up to the test cutoff), 5 node types, 6 foreign keys giving 12 edge types. An edge exists only where the key points to a row in the database, so most attendance rows link to an event but not to a known user. `user_friends` has no timestamp, so friendships are always visible.

higher (AUROC and Precision@13) than one trained on full neighborhoods when little is revealed; **(H3, re-scoped)** retrieval value is concentrated: measured as the AUROC gain per lookup when one table is pulled for every prospect, one or two tables account for most of the gain (previously: search results on `rel-avito`); **(H4, re-scoped)** the AUROC advantage of a batch-aware policy over a per-prospect policy grows with a list's sharing, defined in Section 2 (previously: `rel-event` vs. `rel-avito`); **(H5)** a learned batch value beats the best heuristic (after the milestone).

Status. Everything below ran on `rel-event`: the data pipeline, the partially observed GNN, the lookup environment, five policies and the frontier runner. Three findings stand out. With a quarter of a lookup per prospect, the best simple policy recovers 85% of the AUROC gap between zero lookups and full information (H1). Training on randomly hidden rows makes the model much better when little is revealed (H2). One table, the prospects' own invitation responses, carries nearly all the retrieval value (H3). That explains why researching every prospect a little beats a heuristic that chases shared events, until every prospect has some evidence (H4). One issue is unresolved: a timestamp on the target's own row can postdate the prediction, so comparisons with a strong LightGBM baseline are provisional until we rerun without it (Section 4.1).

2 Data and processing

Dataset and task. We use the `user-ignore` task of `rel-event` from RelBench [6], a public benchmark of real relational databases with temporal splits. The database comes from an event app (the Kaggle Event Recommendation Engine Challenge; downloaded from Hugging Face, no login needed). Users receive invitations to events and respond yes, maybe or no, or never respond. A prediction is made for a user at a time t. The label asks whether the user will ignore more than two invitations in the seven days after t, where ignoring means never responding. The label comes from attendance records after t; the model sees only records dated at or before t. Splits are by time. Training has 19,239 labeled users over 22 prediction dates (16.9% positive). Validation is one day, 2012-11-21, with 2,013 users (11.1% positive): a single day's call list, from which we draw lists of 128. The test day (1,958 users) is held out and will be used once, at the end.

How this stands in for sales. `rel-event` is a public stand-in, not sales data. Users play prospects, events play the accounts and events that prospects share, and the label is an engagement outcome to rank a list by. What carries over is the structure our question depends on: linked tables, entities shared across a list, a fixed lookup budget and a top-K decision. We make no claim that the numbers transfer to a CRM.

From tables to a graph. Following relational deep learning (RDL) [2], every row becomes a node typed by its table, and every primary–foreign-key link becomes an edge in both directions (`f2p_<key>` and its reverse `rev_f2p_<key>`). This gives 5 node types and 12 edge types (Table 1). For example, the attendance row (user 17, event 812, status *invited*, dated 2012-11-14) becomes an `event_attendees` node with two features, its status and its date, and with edges to user 17 and to event 812 in both directions. RelBench proposes a semantic type for every column. Numerical columns are normalized. Categorical columns get learned embeddings. Text columns, such as a user's location, are embedded with 300-d GloVe word vectors [5]. RelBench's `make_pkey_fkey_graph` turns each table into PyTorch Frame tensors; `events`, for instance, be-

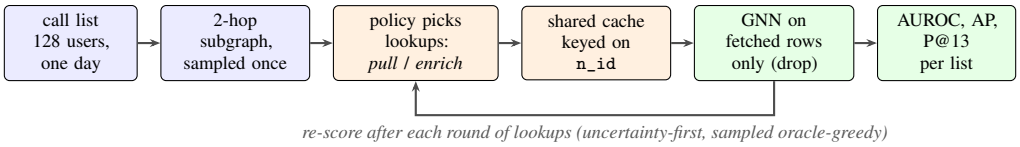


Figure 2: The evaluation pipeline. A call list is sampled once. A policy spends the budget in rounds of $N/4$ paid lookups; an action whose rows are already cached costs nothing. The trained GNN scores the list from the fetched rows, and the metrics are recorded each time the spend reaches a budget.

comes a $2.46\text{M} \times 108$ frame. We cache the result. The first build took 27 minutes and peaked at 30 GB of RAM, and reloading the cache takes about 6 minutes.

Time-respecting sampling. To score a user at time t, PyG's temporal `NeighborLoader` [1] samples up to 128 neighbors per edge type at hop 1 and 64 at hop 2, only among rows dated at or before t. Sampling is disjoint: each target gets its own tree, because two targets can have different prediction times and so may see different parts of the same neighborhood. As a result, an event that two prospects both attended appears twice in a batch, as two separate nodes. The model treats the copies as unrelated, which is fine for prediction. Retrieval must not: an agent that fetches the event once should see it under both prospects. PyG keeps each sampled node's original row id (`n_id`), so the two copies share an id, and our lookup cache is keyed on that id (Section 3.3). The time filter applies to sampled neighbors. A target's own row is always included, which matters for one leakage issue (Section 4.1).

Call lists. A call list is $N = 128$ users drawn at random (seeded) from the validation day. Their neighborhoods are sampled once into one batch, and every policy acts on that same subgraph (Table 2). Every prospect has friendship rows, but only 86 of 128 have attendance rows, 15 have interest clicks and 10 created events, so this list offers 239 pulls (1.9 per prospect). We measure a list's *sharing* per entity type as

$$\text{sharing} = \frac{|\{\text{entities listed by two or more prospects}\}|}{|\{\text{entities listed by any prospect}\}|}. \tag{1}$$

On this list, 930 distinct events sit behind the attendance rows, and 33% of them are listed by two or more prospects (our proposal measured 34%). Of the 1,296 friends who are known users, 11% are shared. The difference between sampled copies and distinct rows matters for cost: a lookup pays for a distinct row once, however many copies the sampler made. Fetching everything costs 2,752 paid lookups per list on average (2,287–3,509 over 100 lists), about 21.5 per prospect.

Table	Sampled rows	Distinct	Role in the lookup environment
`user_friends`	17,630	17,628	hop 1 for all 128 prospects (one pull each)
`event_attendees`	1,647	1,647	hop 1 for 86 prospects (1,519 rows); 128 rows of events prospects created
`event_interest`	116	116	hop 1 for 15 prospects
`events`	1,672	1,060	hop 2: 930 events behind attendance rows (33% listed by 2+ prospects); hop 1: 49 created by 10 prospects
`users`	1,477	1,303	the 128 prospects, and 1,296 friends who are known users (11% listed by 2+)

Table 2: One 128-prospect call list on `rel-event` (`scripts/check_call_list.py`, list seed 0). Sampled rows count every copy the per-target sampler made; distinct rows count each database row once. One forward pass over the whole list takes 0.41 s on an NVIDIA L4.

3 Model design and metrics

3.1 Model

We use RelBench's RDL baseline [2], built in PyG [1]. Each node v has a table type $T(v)$, features x_v (its columns) and a timestamp $\tau(v)$. To score a prospect at time t, each row starts as $h_v^{(0)} = \text{Enc}_{T(v)}(x_v) + \phi(t - \tau(v))$: a per-table PyTorch Frame ResNet over its columns, plus an encoding

ϕ of the row's age at t. Two heterogeneous GraphSAGE layers [3] then update every node:

$$h_v^{(\ell+1)} = \mathrm{ReLU}\Big(\mathrm{LN}\Big(\textstyle\sum_{r\in\mathcal{R}(v)}\big[W_{r,1}^{(\ell)}h_v^{(\ell)} + W_{r,2}^{(\ell)}\sum_{u\in\mathcal{N}_r(v)}h_u^{(\ell)}\big]\Big)\Big). \tag{2}$$

Here a relation r is a (source type, edge type, target type) triple, and $\mathcal{R}(v)$ holds the relations that end at v's type, for example (`event_attendees`, `f2p_user_id`, `users`). $\mathcal{N}_r(v)$ is the set of v's sampled neighbors under r. Each relation has its own weights. $W_{r,2}$ transforms the summed neighbor states and $W_{r,1}$ transforms v's own state, so attendance rows and friendship rows send differently weighted messages. This matches the implementation: PyG's `SAGEConv` keeps a root weight per relation, and `HeteroConv` sums across relations. Layer normalization (LN) rescales each node's vector, which keeps the layer stable when a node has very many neighbors. The neighbor states are summed rather than averaged, because a sum keeps counts, such as how many invitations went unanswered, which a mean or a max discards. GIN [9] shows that a sum followed by an MLP is injective on multisets of countable features. Our layer applies a linear map after the sum, so it is not guaranteed to be injective, but it keeps the count information that mean and max remove.

What two layers compute. Take a user Ana with two attendance rows: a_1 (*invited*, pointing to event e_1) and a_2 (*yes*, pointing to e_2). Layer 1 lets Ana's vector absorb a_1 and a_2, her response history, and lets a_1 and a_2 absorb e_1 and e_2. Layer 2 lets Ana absorb a_1 and a_2 again, now carrying their events. So two layers reach exactly the two hops our lookups act on: a prospect's own rows, and the entities those rows point to. A linear head with batch normalization maps Ana's final 128-d vector to a logit, and $f_\theta(q)$ is its sigmoid. The model has 5.92M parameters. Training minimizes binary cross-entropy with Adam (learning rate 0.005, batch 512, 20 epochs), as in RelBench.

3.2 Partial retrieval (our extension)

The agent sees only O, the rows fetched so far; the prospects' own rows are always in O. We support two ways to hide the rest (Table 3). In *mask* mode an unfetched row's input is replaced by a learned per-table vector:

$$\tilde{h}_v^{(0)} = m_v\, h_v^{(0)} + (1 - m_v)\, e_{T(v)}^{\mathrm{mask}}, \qquad m_v = \mathbb{1}[v \in O]. \tag{3}$$

Revelation is keyed on `n_id`, so fetching a row reveals all its sampled copies at once. We train two regimes. *Full-trained* uses the standard recipe. *Random-trained* draws a share $p \sim U(0,1)$ per batch and reveals each distinct row with probability p, a graph augmentation that shows the model partial views during training. All lookup experiments use drop mode, the mode both regimes were trained in. Mask mode needs its own trained models and is a planned ablation.

	Drop	Mask
What the model knows about an unfetched row	Nothing: not its existence, type or edges	That it exists, its table and its edges; its content is a learned placeholder
Effect on a sum in Eq. 2	One fewer term	Same number of terms, placeholder values
Trade-off	Simple; cannot tell "unread" from "absent"	Keeps structure; needs models trained with placeholders

Table 3: Two ways to hide rows that have not been fetched. All lookup experiments use drop mode.

3.3 Lookup environment

Two actions retrieve rows:

- `pull`(i, T) reveals all of prospect i's sampled rows in table T (up to 128 per relation). On `rel-event` these are attendance records, friendships in either direction, interest clicks, or events the user created. A pull also *lists* the entities those rows point to: their ids become known, but their contents do not.
- `enrich`(T, v) reveals exactly one listed row v, in every sampled copy: an attended event, a friend who is a known user, or an attendance row of an event the user created.

An entity can be enriched only after a pulled row lists it (*list before get*), as with real CRM and enrichment APIs. The accounting is as follows. An action that reveals at least one new row costs

one *paid lookup*. An action whose rows are all cached costs nothing, and each pull can be made once. Budgets and costs always count paid lookups, so a budget of b lookups per prospect means bN paid lookups for the list. *Full retrieval* applies every pull and then enriches until nothing new is listed; its paid cost is the list's *full cost*.

Example. `pull`(Ana, attendance) and `pull`(Ben, attendance) cost one each, and both list event 812. `enrich`(events, 812) costs one and reveals event 812 under both prospects, so both scores change. A second enrich of 812 would cost nothing.

The objective and the batch value. For a list $Q = \{q_1, \ldots, q_N\}$ with labels y_i, a policy π chooses lookups in sequence. With O_B^π its fetched rows after B paid lookups and M a ranking metric, we want $\max_\pi \mathbb{E}\left[M(f_\theta(Q \mid O_B^\pi), y)\right]$. Ranking metrics are piecewise constant in the scores, so they don't say how much one lookup helped. We therefore score a lookup a by its *batch value*, the list-wide drop in cross-entropy ℓ:

$$V(a \mid O) = \sum_{i=1}^{N} \left[\ell\big(f_\theta(q_i \mid O), y_i\big) - \ell\big(f_\theta(q_i \mid O \cup \{a\}), y_i\big)\right]. \tag{4}$$

Cross-entropy is a smooth, per-prospect surrogate, and summing it over the list credits a shared entity for every prospect it informs. It measures retrieval value, not decision value: lowering the loss of a prospect far from the cutoff raises V without changing who gets called. Section 4.3 shows that this matters. V needs labels, so for now we use it only in a reference policy; after the milestone a second model will learn to predict it (H5).

3.4 Policies

Every policy proposes the next lookups for the whole list in rounds of $N/4 = 32$ paid lookups. None of the five is learned. **Random** picks uniformly among legal lookups; it is the floor. **Case-local** lets the least-researched prospect take its own next lookup: its next table, then the entity its rows mention most. Tables are taken in alphabetical order, which on `rel-event` puts `event_attendees` first; Section 4.4 shows that this ordering matters. **Uncertainty-first** does the same with prospects ordered by $p(1-p)$, re-scored each round, as in per-instance active feature acquisition [4, 7]. **Coverage** is batch-aware: it takes the lookups that inform the most prospects first. A listed entity informs every prospect whose pulled rows point to it; a pull informs one. **Sampled oracle-greedy** is a label-informed reference. Each round it samples 64 legal lookups, computes their true V with the labels, and takes the best. It is not deployable, and it is not a guaranteed upper bound: it sees only 64 candidates per round and plans greedily.

3.5 Metrics

E1 reports RelBench's official validation AUROC on all 2,013 validation users, as mean $\pm$ s.d. over three seeds. Lookup experiments report, per call list, AUROC, average precision (AP) and Precision@K: the share of true positives among the $K = \lceil 0.1N \rceil = 13$ highest-scored prospects. A random ranking scores the positive rate, about 0.11. We average over lists and give 95% percentile-bootstrap intervals over lists (2,000 resamples). Lists, not prospects, are the units, because prospects on one list share entities. Every policy runs on the same lists and subgraphs in one run, so comparisons are paired. Two limitations apply. The intervals cover list-to-list variation for one trained model on one day, not training-seed variation. And lists are drawn from the same 2,013 users, so each user appears on about six of 100 lists; the lists are not fully independent, and the intervals are likely somewhat narrow.

4 Results so far

All numbers are on the `rel-event` validation day (2012-11-21) unless marked; `rel-f1` is the development dataset. Runs: one NVIDIA L4 GPU on AWS, Nov 7–8.

4.1 Validity checks

Implementation. Eleven unit tests pass on the laptop and the GPU machine. They cover the per-list metrics, temporal sampling, the environment's guarantees (full retrieval equals the full model, cache hits are free, list before get), and that every policy spends exactly its budget in paid lookups using

legal actions. On both datasets, `check_reveal.py` found that random revelation never splits the copies of a shared row, and that with every row revealed both drop and mask mode reproduce the full model exactly. `check_lookup.py` confirmed on both datasets that zero lookups reproduce the model's $p = 0$ scores and full retrieval reproduces its full-information scores. On one `rel-event` list, full retrieval took 2,555 paid lookups and revealed every sampled row.

Temporal leakage. `check_leakage.py` found no sampled neighbor dated after its target's prediction time: none among 431,731 timestamped rows on `rel-f1`, and none among 145,217 on `rel-event` (all four timestamped tables, 5 training and 5 validation batches). It did flag target rows. RelBench's task table includes users before their recorded `joinedAt`: 35% of training targets and 52 of 2,013 validation targets. A target's own row is always in its subgraph, so every model sees that `joinedAt`, LightGBM and RelBench's baselines included, and it leaks a little: those 52 validation users are 17% positive against 11% overall. All comparisons with LightGBM are therefore provisional until we rerun without `joinedAt`, and the GNN numbers may shift slightly too.

Why attendance history is not the label. Attendance rows are dated by their event's start time. A prediction at t therefore sees only invitations to events that had started by t, while the label concerns invitations in the week after t, whose rows are never sampled. One assumption remains: a row's status is the database's final snapshot, so a response given after t to an earlier invitation would be visible. RelBench makes the same assumption. We expect such late responses to be rare, but we have not measured how rare.

4.2 Training on partial views, and the decision gate (E1, E2)

Table 4 reports validation AUROC when each distinct sampled row is revealed with probability p. The full-trained GNN scores 0.818 ± 0.030 with every row and 0.612 ± 0.046 with none. The random-trained model has a higher mean at every p, and the difference is largest when little is revealed: 0.136 at $p = 0$ (0.748 vs. 0.612) and 0.062 at $p = \frac{1}{4}$. With three seeds per regime, the first difference is well beyond the seed spread (Welch $t \approx 4.9$) and the second is suggestive ($t \approx 2.3$). At $p \geq \frac{1}{2}$ the differences are within seed variation. Precision@13 tells the same story. From $p = 1$ to $p = 0$, the full-trained model drops from 0.563 to 0.166 and the random-trained model from 0.574 to 0.265.

Dataset	Training	$p = 1$	$p = 0.75$	$p = 0.5$	$p = 0.25$	$p = 0$
`rel-event`	full	0.818 ± 0.030	0.818 ± 0.035	0.815 ± 0.039	0.794 ± 0.045	0.612 ± 0.046
`rel-event`	random	0.853 ± 0.020	0.859 ± 0.017	0.862 ± 0.015	0.856 ± 0.013	0.748 ± 0.015
`rel-f1` (dev)	full	0.773 ± 0.027	0.778 ± 0.027	0.772 ± 0.028	0.763 ± 0.025	0.655 ± 0.031
LightGBM on the prospect's own row only (provisional; see Section 4.1): `rel-event` 0.853, `rel-f1` 0.661						

Table 4: Validation AUROC on all validation targets, mean $\pm$ s.d. over 3 training seeds, when each distinct sampled row is revealed with probability p (hidden rows dropped). Full: standard training. Random: a share $p \sim U(0, 1)$ revealed per training batch.

Why training on partial views helps. The likely mechanism is a match between training and inference. A full-trained model only ever saw complete neighborhoods, so when most rows are hidden its inputs look unlike anything it was trained on. A random-trained model has seen near-empty and half-empty neighborhoods, and it learned to rely on whatever is present, including the prospect's own row. This reverses our `rel-f1` result from the proposal, and we do not yet know why. `rel-f1` has far fewer training targets (1,353 vs. 19,239) and a different neighborhood structure, and either could matter, but we have not tested them.

A strong but provisional baseline. LightGBM on the prospect's own row (locale, birthyear, gender, joinedAt, location, timezone) reaches 0.853 validation AUROC. That equals the random-trained GNN with every row and beats the full-trained GNN. On call lists it scores AP 0.568 and Precision@13 0.545, slightly below the full-information GNN (about 0.58–0.60 and 0.56–0.57). Two readings are possible. Profile features may carry much of the AUROC signal on this day, with boosted trees using them better than the GNN's row encoder. Or the `joinedAt` leak may help the trees more than the GNN. The rerun without `joinedAt` will separate the two, and it will also show how much of the GNN's own signal comes from attendance histories. Until then we treat LightGBM

as a reference line, not a benchmark. The leaderboard's test numbers in our proposal (GNN 0.804, LightGBM 0.773) come from a different day and are not comparable.

E2: the decision gate. A lookup policy moves a list somewhere between zero lookups and full information, so that gap bounds what any policy can show. If it were tiny, every policy would look the same. Before running frontiers we agreed with our mentor to proceed only if $\Delta = \text{AUROC}(p{=}1) - \text{AUROC}(p{=}0) \geq 0.02$ for the full-trained model. Seed-to-seed spread on `rel-f1` was 0.02–0.05, so a smaller gap could not separate policies. On `rel-event`, $\Delta = 0.818 - 0.612 = 0.206$, so `rel-event` remains the dataset of record and `rel-avito` a stretch goal.

4.3 Budget frontiers (E3)

Figure 3 runs all five policies on the same 100 call lists, at budgets from $\frac{1}{4}$ to 16 paid lookups per prospect. The lists are scored by the seed-0 full-trained model, chosen before any frontier ran. We summarize a policy at budget b by the share of the gap it recovers,

$$\text{gap recovered}(b) = \frac{M_{\text{policy}}(b) - M_{\text{zero}}}{M_{\text{full}} - M_{\text{zero}}}, \tag{5}$$

where M_{zero} is a list's score with only the prospects' own rows and M_{full} its score after full retrieval.

H1: concave frontiers. With 32 paid lookups for 128 prospects ($b = \frac{1}{4}$, about 1.2% of the cost of full retrieval), case-local reaches 0.752 AUROC per list, against 0.597 with zero lookups and 0.780 with full information. That is $(0.752 - 0.597)/(0.780 - 0.597) = 85\%$ of the gap. An earlier, independently sampled run on 30 lists gave 81%. From $b = \frac{1}{2}$, case-local exceeds full information (0.802 vs. 0.780). On Precision@13 it reaches 0.542 at $b = 1$, against 0.549 with everything fetched. At $b = 16$, every heuristic except random is within 0.003 of full information. That is expected: at large budgets all policies have revealed nearly the same rows.

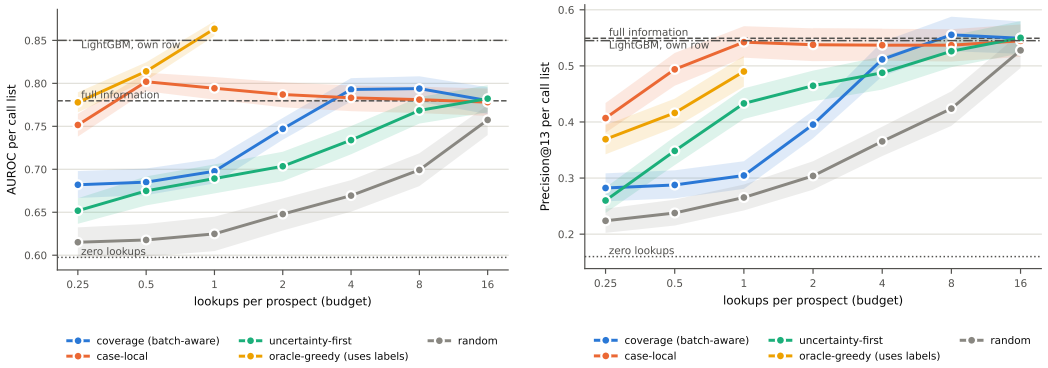
These zero and full values differ from E1's (0.612 and 0.818) for two reasons. E3 uses only the seed-0 model, the weakest of the three (0.784 on the full validation set). And it averages AUROC within 128-prospect lists instead of computing it once over all 2,013 users.

The policies differ by more than their intervals. Paired with case-local on the same lists at $b = 1$, coverage is -0.097 AUROC (95% interval $[-0.114, -0.078]$), uncertainty-first -0.105 $[-0.119, -0.091]$ and random -0.169 $[-0.192, -0.148]$. Sampled oracle-greedy is $+0.069$ $[0.055, 0.084]$.

Why coverage loses early and wins later. Coverage maximizes *entity-level coverage*: how many prospects each lookup informs. A ranking also needs *prospect-level coverage*: how many prospects have any evidence at all. The two pull in different directions. In its first round, coverage has nothing listed yet, so it pulls 32 prospects. From then on, any event shared by two of those 32 informs two prospects, so it outranks every remaining pull, which informs one. By $b = 1$, coverage has made 87 event enrichments and only 16 attendance pulls, and most prospects have no evidence. We call this *starvation*. Case-local does the opposite. Its first lookup for almost every prospect is the attendance table: 28.9 of its first 32 lookups, and by $b = 1$ all of the roughly 90 prospects that have attendance rows. Uncertainty-first starves in a milder way, returning to the same uncertain prospects. Once every prospect has evidence, shared entities become the better buy: at $b = 4$ and $b = 8$, coverage edges ahead of case-local ($+0.010$ $[0.001, 0.019]$ and $+0.013$ $[0.009, 0.016]$). Together these results suggest a two-stage strategy: first give every prospect a minimum of evidence, then spend on shared entities. That is the breadth-then-share heuristic we test next (Section 5).

Retrieval value is not decision value. Sampled oracle-greedy picks lookups by their true V. It gains $+0.069$ AUROC over case-local at $b = 1$ but loses on Precision@13 (0.490 vs. 0.542). Lowering the list's cross-entropy improves the ranking overall, but it does not necessarily put the right prospects in the top 13. This is direct evidence that the surrogate in Eq. 4 and the decision a rep makes can diverge, and it motivates a cutoff-aware value for H5.

H2 under chosen lookups. Figure 4 repeats the frontier with the seed-0 random-trained model on 30 lists. This is a separate run, so it is not paired with Figure 3. The random-trained model starts much higher with zero lookups, 0.792 against 0.597. With case-local it reaches 0.866 at $b = 4$, above its own full information (0.852) and above the provisional LightGBM reference (0.850). With this model, the policy order changes at small budgets. Uncertainty-first and coverage beat case-local at



(a) AUROC per call list (b) Precision@13 per call list

Figure 3: Budget frontiers on `rel-event`: mean metric per 128-prospect call list against paid lookups per prospect (log scale), 100 lists, seed-0 full-trained model; bands are 95% bootstrap intervals over lists. Dotted: zero lookups. Dashed: full information (on average 2,752 paid lookups per list). Dash-dot: LightGBM on the prospect's own row, provisional (Section 4.1), scored on 200 random lists of the same day. Oracle-greedy is the sampled, label-informed reference policy and is run only to $b = 1$.

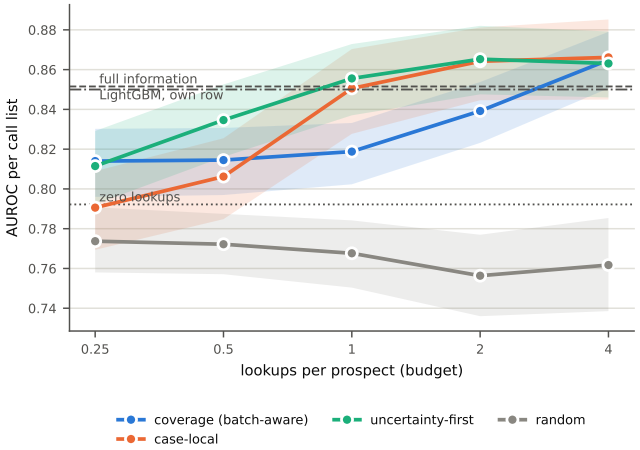


Figure 4: The AUROC frontier with the random-trained model (seed 0): 30 call lists, four policies, 95% bootstrap bands. Dotted: zero lookups (0.792). Dashed: full information (0.852). Dash-dot: LightGBM on the prospect's own row (0.850, provisional). A separate run from Figure 3, so the two are not paired list by list.

$b = \frac{1}{4}$ (paired $+0.021$ $[0.004, 0.037]$ and $+0.023$ $[0.016, 0.031]$), and uncertainty-first also leads on Precision@13 (0.456 vs. 0.387). Case-local catches up once every prospect's attendance has been pulled ($b = 1$: 0.851). Under these conditions, the model's uncertainty became a useful guide only when the model had been trained on partial views.

4.4 What is worth a lookup (E4)

Table 5 pulls one table for every prospect, then enriches everything those rows list, and measures the AUROC gain over zero lookups. Under the current model and training regime, one table carries the retrieval value. Pulling attendance histories (89 paid lookups per list) raises AUROC by 0.224 $[0.188, 0.261]$, and enriching the roughly 1,000 events they list adds only 0.002 more. Interest clicks change nothing. Pulling the events a prospect created lowers AUROC by 0.029, and pulling friendships lowers it by 0.062, both with intervals that exclude zero.

Attendance rows hold each past invitation's response (invited, yes, maybe, no), the most direct evidence of whether a user answers invitations. Section 4.1 explains why they do not reveal the future label. The negative values are not evidence that friendships carry no signal. They show that this model, trained with every row present, does not use friendship and created-event rows well on

this day. The extra rows may add noise, or 128 featureless friendship rows may swamp the summed message at layer 2; separating these needs the ablations in Section 5.

This also explains two earlier observations. Case-local leads at small budgets because its first lookup per prospect is the valuable one, and partial retrieval can beat full retrieval because full retrieval adds rows that this model handles poorly. Case-local reaches attendance first because it takes a prospect's tables in alphabetical order, and `event_attendees` sorts first. Its result therefore reflects an interaction between lookup order and where the value sits across tables. A case-local with a value-aware table order is the fair version to compare against.

Table pulled for every prospect	Pull only			Pull + enrich	
	lookups	AUROC gain [95% interval]	per 100	lookups	gain
`event_attendees`	89	$+0.224$ $[+0.188, +0.261]$	$+0.251$	1,122	$+0.226$
`event_interest`	11	$+0.000$ $[-0.018, +0.019]$	$+0.004$	93	$+0.002$
`events` (created)	7	-0.029 $[-0.046, -0.014]$	-0.395	183	-0.033
`user_friends`	128	-0.062 $[-0.107, -0.016]$	-0.048	1,447	-0.089

Table 5: Value of each table: AUROC gain over zero lookups when one table is pulled for every prospect (and, right, when everything those rows list is enriched too), mean over 30 call lists, seed-0 full-trained model (`scripts/table_value.py`). "Per 100": gain per 100 paid lookups. A first run on 10 lists gave the same signs for attendance, created events and friendships.

4.5 Sharing and batch-awareness (E5)

We split the 100 lists into thirds by sharing (Eq. 1, over all entity types) and compare coverage with case-local, averaging the paired AUROC difference over $b \leq 2$. Coverage's deficit grows with sharing: -0.067 $[-0.087, -0.049]$ on the least-shared third (11–16% shared), -0.076 $[-0.097, -0.056]$ on the middle third (16–19%), and -0.099 $[-0.118, -0.080]$ on the most-shared third (19–24%). The paired strata are the main evidence; the per-list correlation between sharing and coverage's advantage is weak (-0.17). The mechanism matches E3. On lists that share more, more of coverage's budget goes to events shared among the prospects it pulled first, and more prospects are left without evidence. At small budgets, then, this heuristic contradicts H4. That is a finding about coverage's starvation, not about batch-aware retrieval in general. The test is also limited: sharing ranges only from 11% to 24% across random lists of one day.

Summary against the hypotheses. H1: supported (85% of the gap at about 1% of the cost, replicated on a separate 30-list run). H2: supported under random revelation when little is revealed, and under chosen lookups (the random-trained frontier starts at 0.792 and reaches 0.866). H3 (re-scoped): supported for this model (attendance histories carry the retrieval value; friendships and created events lower AUROC). H4 (re-scoped): not supported at small budgets by the one batch-aware heuristic tested, which starves prospects, and more so on lists that share more; a small batch-aware advantage appears at $b \geq 4$. H5: next.

5 Next steps

The results change what the learned policy must do. It has to cover every prospect first, know which table is worth a lookup, and value moving prospects across the call cutoff rather than only lowering loss. Table 6 orders the remaining work by priority.

The learned policy (H5). E3's oracle result is the motivation: a value that only lowers loss does not target the decision. On training-day call lists, where labels are available, we will run the existing policies to reach a variety of states, then compute the true V of sampled candidate lookups. We will also compute a cutoff-aware variant that weights each prospect's loss change by its closeness to the top-13 boundary. We then train a model to predict these values without labels: first from hand-built features of the lookup and the prospects it touches, then with a GNN on the revealed graph, using a list-level head and a singleton head as in per-node query valuation [10]. At test time the policy takes the highest-valued lookup, re-scores and repeats, in the same frontier runner as the heuristics.

Deliverables (to Dec 10). A Colab that rebuilds the graph, trains the model and draws the frontiers on a small budget, and a Medium post built around Figures 1 and 3.

	Experiment	Why	When
1	Remove `joinedAt`; rerun E1, LightGBM and the main frontier	Establish validity	Nov 12–18
2	Breadth-then-share heuristic	Fixes starvation; tests the two-stage strategy	Nov 12–18
3	Case-local with value-aware and randomized table order	Tests whether lookup order explains its lead	Nov 12–18
4	Learned, cutoff-aware lookup value (H5)	Aligns the policy with the top-13 decision	Nov 19–30
5	Full- vs. random-trained frontiers, paired, more seeds	Isolates the effect of partial-view training	Nov 19–30
6	Mask vs. drop mode	Tests representing unfetched rows	Nov 19–30
7	Mean aggregation, GAT [8], LightGBM score as a GNN input	Do the findings depend on the GNN?	late Nov
8	Larger lists ($N \in \{32, 128, 512\}$), test day, `rel-avito`	External validity	Dec

Table 6: Remaining work, in priority order.

References

[1] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[2] Matthias Fey, Weihua Hu, Kexin Huang, Jan Eric Lenssen, Rishabh Ranjan, Joshua Robinson, Rex Ying, Jiaxuan You, and Jure Leskovec. Position: Relational deep learning – graph representation learning on relational databases. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 13592–13607, 2024.

[3] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems 30*, 2017.

[4] Alexander Luke Ian Norcliffe, Changhee Lee, Fergus Imrie, Mihaela van der Schaar, and Pietro Liò. Stochastic encodings for active feature acquisition. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 46784–46814, 2025.

[5] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.

[6] Joshua Robinson, Rishabh Ranjan, Weihua Hu, Kexin Huang, Jiaqi Han, Alejandro Dobles, Matthias Fey, Jan E. Lenssen, Yiwen Yuan, Zecheng Zhang, Xinwei He, and Jure Leskovec. RelBench: A benchmark for deep learning on relational databases. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024) Datasets and Benchmarks Track*, 2024.

[7] Michael Valancius, Maxwell Lennon, and Junier Oliva. Acquisition conditioned oracle for nongreedy active feature acquisition. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 48957–48975, 2024.

[8] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

[9] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019.

[10] Zihe Zhou. Budgeted task-aware acquisition of dynamic networks. *arXiv preprint arXiv:2609.05862*, 2026.